

P0502r0

Throwing out of a parallel algorithm terminates—but how?

Published Proposal, 9 November 2016

This version:

<http://wg21.link/P0502r0>

Authors:

[JF Bastien](#) (Apple)

[Bryce Adelstein Lelbach](#) (Lawrence Berkeley National Laboratory)

[H. Carter Edwards](#) (Sandia National Laboratory)

Audience:

SG1, LWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

github.com/jfbastien/papers/blob/master/source/P0502r0.bs

Abstract

The Committee approves of terminating if exceptions leave parallel algorithms, but where to mandate termination should be updated.

Table of Contents

1	Background
1.1	Straw Polls
1.2	Action
2	Proposed Wording
3	Acknowledgement
	References
	Informative References

§ 1. Background

The Standard was simplified in [\[P0394r4\]](#): exceptions leaving parallel algorithms lead to `std::terminate()` being called. This matches the behavior of exceptions leaving `main()` as well as `std::thread()`.

The following National Body comments from [\[P0488R0\]](#) were discussed in SG1 at Issaquah, along with [\[p0451r0\]](#):

- US 15, US 167: Don't `terminate()` when a parallel algorithm exits via uncaught exception and either re-add `exception_list`, add `noexcept` policies + re-add `exception_list`, make it UB or throw an unspecified exception (revert [\[P0394r4\]](#)).

- US 17, US 169: Don't `terminate()` when a parallel algorithm exits via uncaught exception and re-add `exception_list` (revert [P0394r4]).
- US 16, US 168: Clarify which exception is thrown when a parallel algorithm exits via uncaught exception.
- US 170: Add a customization point for `ExecutionPolicy`s which defines their exception handling behavior (don't re-add `exception_list`).
- CA 17: Preserve the `terminate()`-on-uncaught-exception behavior in the parallel algorithms (keep [P0394r4]).

§ 1.1. Straw Polls

The following straw polls were taken:

Straw Poll A: In 25.2.4 §2, have uncaught exception behavior be defined by `ExecutionPolicy`. In 20.19 define the behavior for the three standard policies in C++17 (`seq`, `par`, `par_unseq`) as `terminate()`.

<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>Many</i>	<i>7</i>	<i>1</i>	<i>1</i>	<i>0</i>

⇒ Consensus to write a paper for this before the end of the week. Bryce, JF, and Carter will write it.

Straw Poll B: Do we want to rename the policies to reflect the fact that they call `terminate()` instead of throwing exceptions.

<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>1</i>	<i>7</i>	<i>9</i>	<i>6</i>	<i>7</i>

⇒ No consensus for change.

Straw Poll C: Beyond the changes from the first straw poll, additional changes are required.

<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>2</i>	<i>0</i>	<i>10</i>	<i>11</i>	<i>6</i>

⇒ No consensus for change.

§ 1.2. Action

This paper follows the guidance from *straw poll A*: there is no behavior change, but the behavior is specified to allow future execution policies which exhibit different behavior.

§ 2. Proposed Wording

Apply the following edits to section 15.5.1 §1 note, bullet 1.13:

15.5.1 The `std::terminate()` function [except.terminate]

1. In some situations exception handling must be abandoned for less subtle error handling techniques. [*Note*: These situations are:

[...]

(1.13) — for parallel algorithms whose `ExecutionPolicy` specify such behavior (20.19.4, 20.19.5, 20.19.6), when execution of an element access function (25.2.1) of a parallel algorithm exits via an exception (25.2.4), or

[...]

— *end note*]

Apply the following edits to section 20.19:

20.19.4 Sequential execution policy [execpol.seq]

```
class execution::sequenced_policy { unspecified };
```

1. The class `execution::sequenced_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and require that a parallel algorithm's execution may not be parallelized.
2. During the execution of a parallel algorithm with the `execution::sequenced_policy` policy, if the invocation of an element access function exits via an uncaught exception, `terminate()` shall be called.

20.19.5 Parallel execution policy [execpol.par]

```
class execution::parallel_policy { unspecified };
```

1. The class `execution::parallel_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized.
2. During the execution of a parallel algorithm with the `execution::parallel_policy` policy, if the invocation of an element access function exits via an uncaught exception, `terminate()` shall be called.

20.19.6 Parallel+Vector execution policy [execpol.vec]

```
class execution::parallel_unsequenced_policy { unspecified };
```

1. The class `execution::parallel_unsequenced_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized and vectorized.
2. During the execution of a parallel algorithm with the `execution::parallel_unsequenced_policy` policy, if the invocation of an element access function exits via an uncaught exception, `terminate()` shall be called.

Apply the following edits to section 25.2.4 [algorithms.parallel.exceptions] §2:

During the execution of a parallel algorithm, if the invocation of an element access function exits via an uncaught exception, the behavior is determined by the `ExecutionPolicy`. ~~`terminate()` is called.~~

Thank you to all SG1 participants: David Sankel, Alisdair Meredith, Hartmut Kaiser, Pablo Halpern, Jared Hoberock, Michael Wong, Pete Becker. Special thanks to the scribe Paul McKenney.

§ References

§ Informative References

[P0394r4]

JF Bastien, Bryce Adelstein Lelbach. [Hotel Parallelifornia: terminate\(\) for Parallel Algorithms Exception Handling](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0394r4.html). 23 June 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0394r4.html>

[P0451R0]

Bryce Adelstein Lelbach, Alisdair Meredith. [Future-Proofing Parallel Algorithms Exception Handling](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0451r0.html). 14 October 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0451r0.html>

[P0488R0]

Barry Hedquist. [WG21 Working paper: NB Comments, ISO/IEC CD 14882](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0488r0.pdf). 18 October 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0488r0.pdf>